

SkyGuard As-Built

The brief, the mathematics, and what is actually running.

The blueprint describes an intended system and carries the reasoning that got us there. This document describes the **built** one, and shows the arithmetic underneath it in full — every formula with a worked example you can check by hand. Where the two documents disagree, this one is right: it was written by reading the code and querying the deployed service. Every number came from `/api/summary`, a model report, or a calculation reproduced from the source.

CONTENTS

I · THE BRIEF

- 01 The problem statement
- 02 How it is scored
- 03 End-to-end workflow

II · THE MATHEMATICS

- 04 The physics you have
- 05 Why three parameters
- 06 Step 1 — anomaly
- 07 Step 2 — neighbours
- 08 Step 3 — sigma from MAD
- 09 Step 4 — z and the bands
- 10 Two passes
- 11 What it cannot see
- 12 Alerts as episodes

III · THE DATA

- 13 The network we made
- 14 Fault models
- 15 Real vs. generated

- 16 How accuracy is measured
- 17 Measured results
- 18 The learned model

IV · AS-BUILT

- 19 Shape of the system
- 20 The live node
- 21 Ingest and storage
- 22 Module inventory
- 23 API surface
- 24 Frontend
- 25 Deployment
- 26 Not built
- 27 Traps already paid for
- 28 Running it

PART I – THE BRIEF

01 The problem statement

Detect abnormal, inconsistent or faulty observations from Automatic Weather Stations in real time, using **only** Temperature (°C), Atmospheric Pressure (hPa) and Relative Humidity (%). Distinguish genuine meteorological events from sensor/data anomalies while minimising false alarms and enabling scalable deployment across large networks.

REQUIRED	DETAIL
Objectives	Real-time detection · identify sensor faults, spikes, frozen values, communication errors · learn normal temporal and seasonal patterns · multivariate consistency analysis · confidence scores and explainable reasoning · predict sensor degradation and maintenance needs · optionally suggest corrected values.
Outputs	Real-time alerts · severity and confidence scores · root-cause classification · visualisation dashboard · sensor health status · corrected data (optional).
Deliverable	"Fully executable code with example usage and a document explaining various use cases." Not a prototype and slides.
Evaluated on	Anomaly-injected data.

The one case that matters

The hard failure is drift. A probe that fails slowly reads a little wrong, then a little more, for weeks. No single reading is implausible, so no threshold fires. The failure is not the value — it is the divergence, and divergence is only visible against something that has not diverged. That is the idea the whole system rests on: **compare a station with its neighbours at the same moment**. Weather is shared between nearby stations and cancels in the difference; a fault belongs to one instrument and does not.

CONSEQUENCE

A station with fewer than three usable neighbours cannot be judged this way. The interface labels those NO NEIGHBOURS rather than guessing — visible on the Andaman & Nicobar stations in the station index.

02 How it is scored

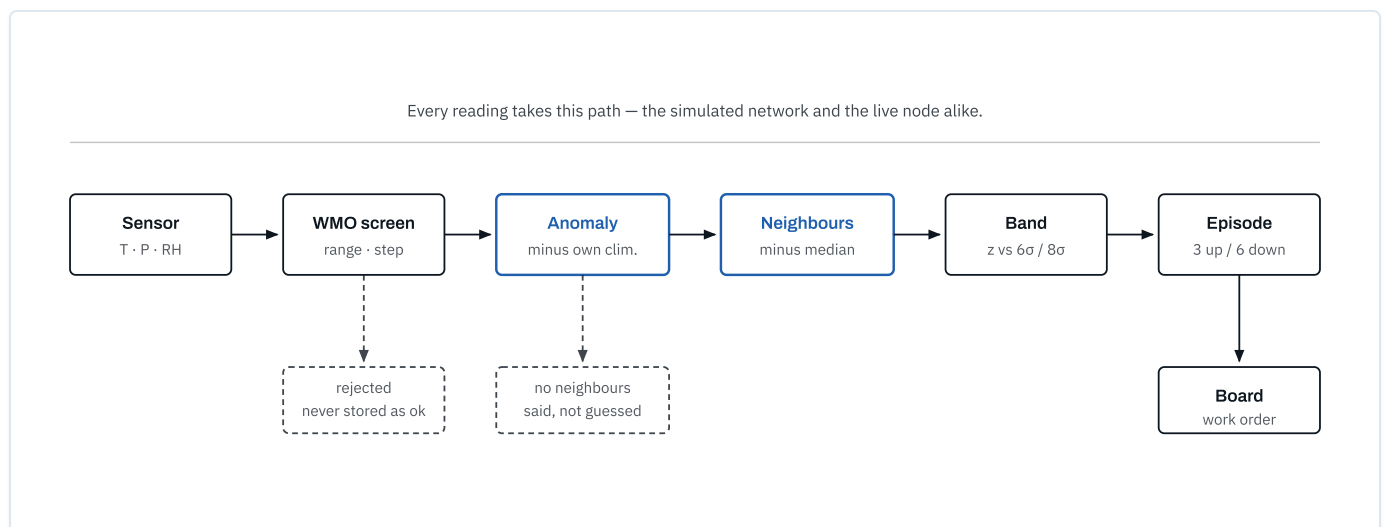
CRITERION	WEIGHT	WHAT EARNS IT HERE
Innovation & Novelty	25%	A signature classifier that names <i>which</i> fault or <i>which</i> event
Detection Accuracy	20%	Neighbour-difference channel; POD-vs-amplitude curves
Real-Time Capability	15%	O(1) streaming state; published latency distributions
Explainability	10%	Uncertainty-propagated physics confidence; exact Shapley
Scalability	10%	Per-station auto-calibrated thresholds
Practical Deployability	10%	Integer-arithmetic edge tier
Visualization / UI	5%	Board, network, judgement screens
Energy Efficiency	5%	Measured uplink reduction

WHAT THIS DRIVES

Innovation (25) + Explainability (10) outweighs Accuracy (20). A classifier that *names* the fault and shows the physical reason scores on all three at once. A bare rarity score scores on one. That is why the pipeline ends in a named band and a named fault, not a probability.

03 End-to-end workflow

One pass, sensor to work order. Everything downstream of the sensor is arithmetic you can check by hand; the rest of Part II does exactly that.



Two branches leave the main line rather than being swallowed: a reading that fails the physics screen is rejected instead of being stored as good, and a station with too few neighbours is reported as unjudgeable instead of scored anyway.

PART II — THE MATHEMATICS

04 The physics you have

Three measured numbers. Everything else is derived from them by thermodynamics, and the derived quantities are more useful than the raw ones because they behave

differently under weather than under failure.

Saturation vapour pressure — the Magnus form

Air can hold only so much water vapour before it condenses, and that ceiling rises steeply with temperature. The Magnus formula gives that ceiling:

$$e_s(T) = 6.112 \cdot \exp\left(\frac{a \cdot T}{b + T}\right) \text{ hPa} \quad \begin{array}{l} a = 17.62, b = 243.12 \text{ over water,} \\ T \geq 0 \text{ }^\circ\text{C} \\ a = 22.46, b = 272.62 \text{ over ice, } T < 0 \text{ }^\circ\text{C} \end{array}$$

The two coefficient sets matter. Using the water set below freezing manufactures spurious violations at high-altitude and northern stations — the formula would claim the air is supersaturated when it is simply cold.

Actual vapour pressure and dew point

Relative humidity is a *percentage of that ceiling*, which is why RH alone tells you little: 60% at 30 °C is far more water than 60% at 5 °C. Multiply to get the actual amount, then invert to get the temperature at which that amount would saturate — the dew point.

$$e = (RH / 100) \cdot e_s(T) \text{ hPa} \quad \gamma = \ln(RH/100) + a \cdot T / (b+T) \quad T_d = b \cdot \gamma / (a - \gamma) \text{ }^\circ\text{C}$$

WORKED — $T = 30 \text{ }^\circ\text{C}$, $RH = 60 \%$, $P = 1000 \text{ hPa}$

$$e_s = 6.112 \cdot \exp(17.62 \cdot 30 / (243.12 + 30)) \quad 42.3372 \text{ hPa}$$

$$e = 0.60 \cdot 42.3372 \quad 25.4023 \text{ hPa}$$

$$\gamma = \ln(0.60) + 17.62 \cdot 30 / 273.12 \quad 1.4245$$

$$T_d = 243.12 \cdot 1.4245 / (17.62 - 1.4245) \quad 21.39 \text{ }^\circ\text{C}$$

$$\text{Dew point} \quad 21.39 \text{ }^\circ\text{C}$$

Sanity check by eye: the air is at 30 °C and would need to cool to about 21 °C before dew forms. That is a humid but ordinary tropical afternoon. If the formula had returned 35 °C —

a dew point above the air temperature — that would mean RH above 100%, which is the same statement, not a second independent check.

Specific humidity — this is what pressure is for

Specific humidity is the mass of water vapour per mass of air, in g/kg. Unlike RH, it does **not** change when the air simply warms or cools.

$$q = 0.622 \cdot e / (P - 0.378 \cdot e) \cdot 1000 \text{ g/kg}$$

WORKED — SAME AIR PARCEL

$0.622 \cdot 25.4023$	15.8002
$P - 0.378 \cdot e = 1000 - 9.602$	990.398
$q = 15.8002 / 990.398 \cdot 1000$	15.95 g/kg
Specific humidity	15.95 g/kg

Why this is the useful variable: **q is conserved under dry adiabatic processes and under ordinary diurnal heating.** The sun warms the air, RH falls, and q does not move. So an RH sensor that drifts breaks q, whereas genuine advection — a moist air mass arriving — moves q coherently and moves it at the neighbours too. That is a physical discriminator between fault and weather, available from exactly the three parameters the problem statement allows.

Pressure must be reduced to sea level before comparison

Pressure falls roughly 1 hPa per 8–9 m of altitude. Comparing raw station pressure between neighbours at different elevations is meaningless.

$$P_{\text{MSL}} = P_{\text{station}} \cdot \exp(h / (29.271 \cdot (T + 273.15))) \quad h \text{ in metres}$$

WORKED — TWO NEIGHBOURS, 950 HPA EACH, T = 25 °C

Station A at 500 m	1006.02 hPa
--------------------	-------------

Station B at 600 m

1017.61 hPa

Apparent difference if reduction is skipped

11.59 hPa

Eleven and a half hectopascals of pure artefact from 100 m of hill, against a genuine synoptic pressure anomaly that is often under 5 hPa. Skip this step and every hill station looks broken.

05 Why three parameters are enough

The constraint looks severe until you notice that T, P and RH are not three independent numbers — they are three views of one thermodynamic state, tied together by the relations above. That gives two independent kinds of evidence:

EVIDENCE	QUESTION IT ANSWERS	CATCHES
Cross-channel coherence	Do T, P and RH tell a consistent story <i>at this station</i> ?	A single failed channel: RH drifts while T and P do not, so q breaks while the neighbours' q holds.
Spatial coherence	Does this station agree with its neighbours <i>at this moment</i> ?	Anything that moves one instrument and not the weather — drift, offset, freeze, spike.

The discriminator between a fault and a storm is not magnitude. A genuine squall line produces a large, fast pressure fall — and so does a failing barometer. What separates them is that the squall arrives at the neighbours too, a few minutes earlier or later, and keeps T, RH and P mutually consistent while it does. The fault does neither. **Magnitude is not the signal; coherence is.**

06 Step 1 — anomaly, before anything else

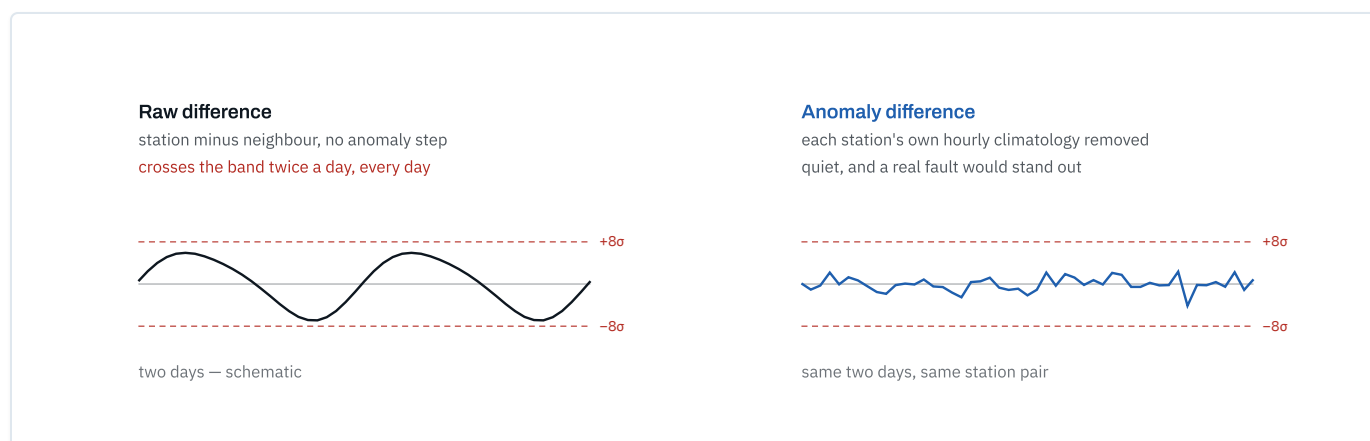
The naive method is to subtract a neighbour's reading from this station's reading. Measured on this network, that flagged **49% of healthy stations**.

The reason is that a station's reading contains its own permanent identity. A mast at 4,196 m is colder than neighbours at 200 m, and — crucially — its *diurnal shape* differs too, because thin air warms and cools faster. So the raw difference swings every single day for reasons that have nothing to do with any instrument.

The fix is to remove each station's own clean-period climatology by hour of day first:

```
# hour of day for every step  $h(t) = (t \cdot 15 \text{ min} / 60) \bmod 24$  # mean of  
each hour over the clean period, per station  $s$   $\text{clim}[h, s] = \text{mean}\{ x(t, s) :$   
 $t < \text{clean\_steps}, h(t) = h \}$  anomaly  $a(t, s) = x(t, s) - \text{clim}[h(t), s]$ 
```

Now every station's anomaly is centred near zero regardless of altitude, and shares the same units and roughly the same daily shape. What remains in `a` is weather plus faults — which is exactly what the difference is meant to compare.



Same station pair, same two days, drawn schematically. The left panel is why differencing raw values flagged 49% of healthy stations: the excursion is the daily cycle of an altitude difference, not a fault.

07 Step 2 — difference against the neighbours

For each station, take the **median** anomaly of its neighbours at the same timestep and subtract it:

```
r(t, s) = a(t, s) - median{ a(t, j) : j ∈ N(s) } N(s) = neighbours of s;  
requires |N(s)| ≥ 3, else the station is not scored
```

Median, not mean, on purpose: the median of five neighbours is unmoved by one of them being broken, while the mean is dragged by it. This is the cheapest robustness in the whole pipeline.

WORKED — ONE TIMESTEP, TEMPERATURE ANOMALIES IN °C

Neighbour anomalies	+0.80 · +0.92 · +0.71 · +1.08 · +0.85
Their median	+0.85
Healthy station reads	+0.90
residual $r = 0.90 - 0.85$	+0.05
Same station with a +2.0 °C offset fault	+2.90
residual $r = 2.90 - 0.85$	+2.05

Read the healthy row carefully: every station in the region is about 0.85 °C warmer than its own climatology — a genuine warm day. The *weather cancelled completely*, leaving 0.05. The faulty row keeps almost the entire 2.0 °C of the fault. That contrast is the entire method.

08 Step 3 — how big is "big"? Sigma from the MAD

A residual of 2.05 °C means nothing until you know the ordinary spread of that station's residuals. A calm inland site might sit within ± 0.2 °C; a coastal site in monsoon might routinely swing ± 1.5 °C. So each station gets its own scale, estimated robustly.

What the MAD is, and where 1.4826 comes from

The Median Absolute Deviation is the median of the distances from the median. Unlike the standard deviation it is not inflated by a handful of extreme values — which matters

enormously here, because the extreme values are the faults we are trying to measure around.

```
m = median(r) MAD = median( |r - m| )  $\sigma$  = 1.4826 * MAD
```

The constant is not arbitrary. For data that really is Gaussian, the MAD comes out at about 0.6745σ — that is the 75th percentile of the standard normal. Dividing by 0.6745 gives 1.4826, so the scaled MAD is directly comparable to a standard deviation. **You can check it: $1 / 0.6745 = 1.4826$.**

WORKED — TEN RESIDUALS, °C

r	0.12 -0.05 0.31 -0.22 0.08 0.44 -0.11 0.02 0.19 -0.30
median m	0.050
r - m	0.07 0.10 0.26 0.27 0.03 0.39 0.16 0.03 0.14 0.35
median of those = MAD	0.150
$\sigma = 1.4826 \cdot 0.150$	0.2224 °C

Shrinkage toward the network

One window of one station is a noisy estimate, and the stations it underestimates are exactly the ones that will cry wolf. So each station's own estimate is pulled halfway toward the network-wide median of all stations' MADs:

```
pooled = medianall stations( MAD )  $sd = \max( 0.5 \cdot MAD + 0.5 \cdot pooled, 1e-6 )$ 
```

WORKED — CONTINUING, WITH A NETWORK POOLED MAD OF 0.42

own estimate	0.2224
pooled	0.4200
$sd = 0.5 \cdot 0.2224 + 0.5 \cdot 0.4200$	0.3212 °C

The window is trailing, lagged, and re-estimated on a stride

This is the correction that mattered most in the whole project.

PARAMETER	VALUE	WHY
SIGMA_WIN	15 days	Long enough for four independent samples given a 2.5-day weather correlation time
SIGMA_LAG	1 day	Holds the reference behind the point being scored, so a slow drift cannot hide inside its own sigma
SIGMA_STRIDE	6 hours	The spread of a weather residual does not move in six hours; exact rolling MAD buys nothing for the cost
MIN_BASE	5 days	Never estimate a scale from less than this

THE BUG THIS REPLACED — MEASURED

Sigma used to be the MAD of the first half of the record, applied to all of it. On a network with *no faults injected at all*:

baseline window |z| p95 = 2.78
scoring period |z| p95 = 4.47 ← inflated 1.61×
false alarms at 4σ: 7.6% of station-hours

Two mistakes, both the same mistake. The estimate was **in-sample** — fitted to the window it was then evaluated on — and it was **fixed**, so a month of changing weather was scored against a fortnight of one regime. A threshold of 4 was really about 2.5 by the end, and that is the entire false-alarm rate. It is also the only version a deployed system could run: at 09:00 you have yesterday, not next week.

Self-masking is the reason for the one-day lag. A drift that has already entered the reference window inflates sigma and thereby hides itself. Holding the reference a day behind buys the detector the beginning of the fault before the fault can start defending itself. This does not solve self-masking for a fault longer than the window — nothing that uses the station's own history can — and that limit is stated wherever the recall figure appears.

09 Step 4 — the z-score and the two bands

```
z(t, s) = ( r(t, s) - m ) / sd |z| ≥ 6 → WATCH WATCH_SIGMA = 6.0 |z| ≥ 8 →  
FAULT FAULT_SIGMA = 8.0
```

```
WORKED - CARRYING BOTH STATIONS THROUGH, SD = 0.3212, M = 0.050
```

```
Healthy: z = (0.050 - 0.050) / 0.3212 0.00 → ok
```

```
Offset fault: z = (2.050 - 0.050) / 0.3212 6.23 → WATCH
```

Six sigma sounds extreme against the usual "3 sigma" intuition, and it is deliberately so. Three sigma on a Gaussian is roughly one exceedance in 370 samples; across 344 stations at 96 steps a day, that is around 90 false flags *per day* from noise alone. Residual distributions also have heavier tails than a Gaussian, which makes it worse. The bands were calibrated empirically against a fault-free control run rather than assumed from theory, which is why they are 6 and 8 rather than 3.

Both constants are named once, in `simulate/faults_and_export.py`, and imported everywhere else. They were previously written in two places, and the map and the alert list disagreed about what was flagged.

10 Two passes — a faulty neighbour poisons the reference

Measured on the first version: healthy stations had `|z| p95 = 2.00` in the training window — exactly what the MAD predicts — and `4.33` afterwards, which flagged half of them. The residual was not drifting. The *reference* was.

A frozen or stepped station still sits inside its neighbours' median, and where a station has only four or five neighbours, one bad member moves that median enough to condemn the rest.

```
pass 1 z1 = grade(all stations) suspect if a station is beyond 3σ for more  
than 20% of the record suspect(s) = mean_t( max_channels |z1(t,s)| ≥ 3 ) >  
0.20 pass 2 z = grade(all stations, excluding suspects from everyone's  
N(s))
```

Note what is *not* consulted: ground truth. The second pass uses only the first pass's own output, which is what a deployed system would have. The ARM path does the same thing.

11 What the detector can and cannot see

The bands and sigma together imply a hard detection floor, and it is worth stating in physical units rather than leaving it implicit.

A drift must accumulate past $6 \cdot \text{sd}$ before it reaches WATCH. With the worked example's sd of 0.3212°C :

DRIFT RATE	REACHES WATCH ($6\Sigma = 1.927^\circ\text{C}$)	REACHES FAULT ($8\Sigma = 2.570^\circ\text{C}$)
$0.15^\circ\text{C} / \text{day}$	12.8 days	17.1 days
$0.05^\circ\text{C} / \text{day}$	38.5 days	51.4 days
$0.0642^\circ\text{C} / \text{day}$	30 days – the whole record	never, in this record

SAY THIS OUT LOUD

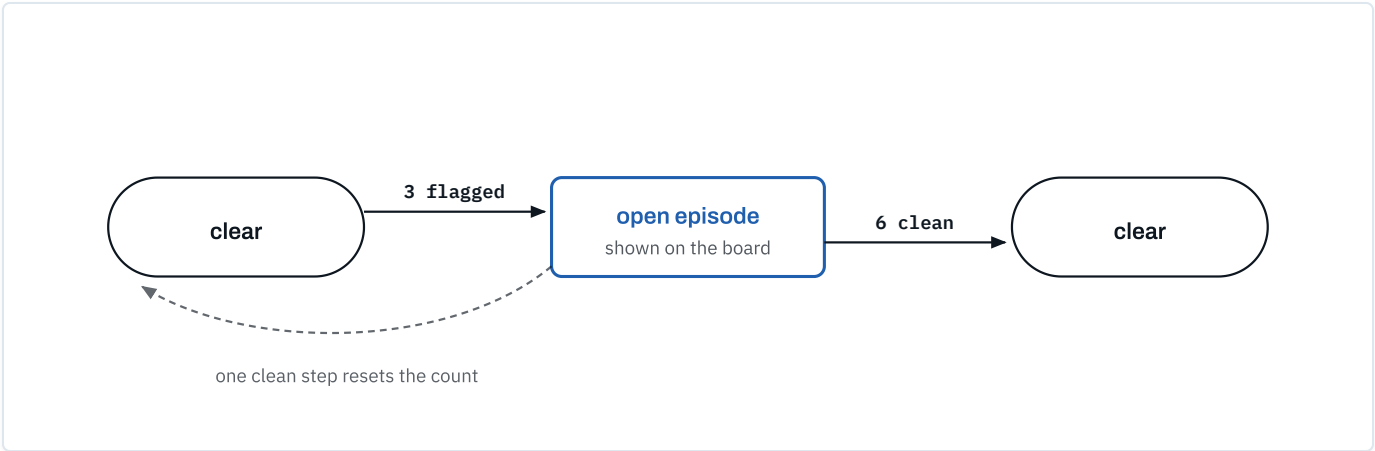
Against a 30-day record and this sigma, **a drift slower than roughly $0.064^\circ\text{C}/\text{day}$ cannot reach the WATCH band at all**, and slower than $0.086^\circ\text{C}/\text{day}$ cannot reach FAULT. It is not that the detector is tuned badly — the evidence is not there yet. A longer record moves this floor down proportionally.

This is also the honest answer to "why is recall 57% and not 95%": the injected drift amplitudes deliberately go down to $0.005^\circ\text{C}/\text{day}$, far below anything a month of data can support. A detector reporting 99% recall on this set would be reporting a bug.

The second blind spot is **dropout**. A station that stops reporting has no value to difference, so it produces no residual and no z at all. It is `NO DATA`, not a fault — which is why the results carry a second recall figure with dropouts excluded.

12 Alerts as episodes, not flags

A grade is recomputed every step with no memory of the last one. Thresholding it directly produced **2,484 flagged runs, half of them a single step long**. An alert therefore has a beginning and an end.



`RAISE_AFTER = 3` , `CLEAR_AFTER = 6` in `web/src/lib/alerts.ts` . Clearing is deliberately slower than raising — otherwise a drifting sensor that wobbles across the band closes its own alert.

PART III – THE DATA

13 The network we made

The generator places stations at **real IMD coordinates** and synthesises a physically coherent weather field over them, then injects faults into a known subset. Because the truth is known by construction, recall and precision can be computed exactly — which is impossible with real IMD data, since no labelled fault list is published.

PARAMETER	VALUE
Stations	344
States & territories	33
Record length	30 days
Step	15 min

PARAMETER	VALUE
Steps per station	2,880
Channels	temp · rh · pres
Record starts	2026-07-25
Stations with an injected fault	28

Station assignment to states is by point-in-polygon against a simplified boundary, with a nearest-polygon fallback for coastal and island sites that fall just outside their polygon. The fallback ranks by *state*, not by polygon — ranking by polygon made an island's runner-up the same state and the ambiguity test always passed.

14 Fault models — exactly what is injected

Seven fault types, each with an amplitude range and duration drawn log-uniformly. Amplitudes deliberately span from *trivially detectable* down to *physically impossible to detect in this record*, so that the result is a detection curve rather than a single flattering number.

TYPE	WHAT HAPPENS TO THE VALUE	WEIGHT
spike	isolated points get <code>±amp</code> ; only those points are marked touched	0.20
frozen	<code>v[:] = v[0]</code> — the last good value repeats exactly	0.18
calibration_drift	<code>v += sign · amp · t / 24</code> — amp is per day	0.17
step_offset	<code>v += sign · amp</code> — a constant shift	0.15
dropout	<code>v[:] = NaN</code> — no observation. Not zero, never imputed	0.12
noise_burst	<code>v += N(0, amp)</code> — variance inflation only	0.10
saturation	pinned at a rail, plus the jitter a stuck ADC still shows	0.08

Amplitude ranges

TYPE	TEMP	RH	PRES
spike	0.5 – 15.0 °C	3 – 60 %	0.5 – 20 hPa
step_offset	0.2 – 5.0 °C	1 – 20 %	0.2 – 8 hPa
noise_burst	0.2 – 4.0 °C	1 – 15 %	0.2 – 5 hPa
calibration_drift	0.005 – 0.15 /day	0.02 – 0.6 /day	0.005 – 0.2 /day

Durations run from 1–3 hours for a spike to 240–1440 hours for a calibration drift. Weights follow real AWS failure logs, where drift and frozen values dominate.

WHY THE RANGES GO DOWN SO LOW

"A detector tuned against 5 K spikes will report 99% recall and miss every real 0.4 K drift." The point of the low end is to make the evaluation produce a *probability-of-detection versus amplitude* curve, which tells you the smallest fault the system can actually find — the number an operator needs — instead of one recall figure that hides it.

One subtlety worth knowing: for spikes, only the displaced samples are marked as touched. Marking the whole window depressed every spike metric in the repo, because a three-hour spike window contains mostly untouched samples that were being scored against a fault that never reached them.

15 Real vs. generated

The first question any reviewer asks, answered before anything else. **Station locations are real IMD sites. The readings are generated.** They will stay generated through the finals — what is being demonstrated is the pipeline that judges them.

PLANE	LOCATIONS	READINGS	FAULT LABELS
Simulated network	real IMD sites	generated	injected — truth known
Live node	real — Gandhinagar	generated, arriving now	injected on demand
WDQMS	real IMD stations	real departures	none published

PLANE	LOCATIONS	READINGS	FAULT LABELS
ARM validation	real — United States	real archive	confirmed by analysts

ARM (US DOE, nine instruments) is the only corpus available with faults a human analyst confirmed. India contributes real station locations, not labelled faults — which is why the detector is *validated* on ARM and *demonstrated* on the Indian network.

16 How accuracy is measured

Every accuracy claim in this project is one of the definitions below. They are not interchangeable, and quoting the wrong one is the easiest way to sound better than you are.

Precision and recall

recall = faults found / faults present did we miss anything? **precision** = true alarms / all alarms is the queue worth working?

WORKED — THE ACTUAL NETWORK RESULT

Stations with an injected fault	28
Stations the detector alerted on	42
Of those, genuinely faulty ($0.38 \cdot 42$)	16
recall = $16 / 28$	0.57
precision = $16 / 42$	0.38

Read together: of 28 real faults we find 16, and to get those 16 an engineer is asked to look at 42 stations. Twenty-six of those visits find nothing. That is the real cost, and it is the number a maintenance planner cares about far more than recall.

Event recall vs. point recall

These answer different questions and usually differ a lot.

MEASURE	COUNTS	TELLS YOU
Point recall	individual flagged samples inside faults	whether you catch the <i>bulk</i> of a fault
Event recall	an event is a hit if any sample in it is flagged	whether you catch the <i>onset</i> at all

High event recall with low point recall means the system notices faults but does not track them; the reverse means it is good at long obvious faults and misses short ones. The gap between the two is the diagnostic.

Average precision

Precision and recall both depend on where you put the threshold, so a single pair of numbers describes one operating point, not the detector. Average precision integrates precision over all recall levels:

$$AP = \sum_k (\text{recall}_k - \text{recall}_{k-1}) \cdot \text{precision}_k$$

Compare AP against the **base rate** — the fraction of samples that are positive. A detector with AP equal to the base rate has learned nothing; the ratio between them is the *lift*, and that is the honest headline for a classifier.

THE OPERATING POINT IS A CHOICE, NOT A RESULT

This same detector reaches **86% recall at 15% precision** if the bands are loosened. Neither setting is "the accuracy". We publish 57/38 because a queue nobody trusts is worse than a shorter one — but the trade is a dial, and an operator with more field crews should turn it.

17 Measured results

From `/api/summary` on the deployed service — 28 faults injected into a 344-station network, with the grader given the readings and nothing else.

57% RECALL of injected faults found	38% PRECISION of alerting stations	72% RECALL EXCL. DROPOUTS silence has no residual	73 ALERT EPISODES 42 stations alerting
--	---	---	---

The dropout-excluded figure jumps from 57% to 72% because a station that stops reporting produces nothing to difference. Counting those as detection failures blames the detector for a communications outage, so both numbers are published and the reason is stated with them.

18 The learned model

A `HistGradientBoostingClassifier` trained on the ARM corpus with `LeaveOneGroupOut` by station — so it is always tested on an instrument it never saw — served at `/api/model/score` with TreeSHAP attributions. The deployable variant drops the three housekeeping features (panel voltage, logger temperature) that a three-parameter AWS will not have.

FEATURE GROUP	FEATURES
Residual now	<code>z</code> , <code>abs_z</code>
Residual change	<code>z_d1</code> , <code>z_d6</code>
Residual window	<code>z_max_24</code> , <code>z_mean_24</code>
Belief state	<code>bias_sigma</code> , <code>drift_sigma_day</code>
Behaviour	<code>flat_frac_24</code> , <code>gap_frac_24</code> , <code>rail_near</code>
Excluded (housekeeping)	<code>hk_volt_z</code> , <code>hk_ltemp_z</code> , <code>hk_max_abs_z</code>

0.406 AVG PRECISION pooled, 46,212 rows	0.331 BASE RATE 15,316 positives	1.22× LIFT over base rate	≈0.000 RECALL @ 80% PREC 0.00026 — effectively none
--	---	--	--

READ THIS HONESTLY

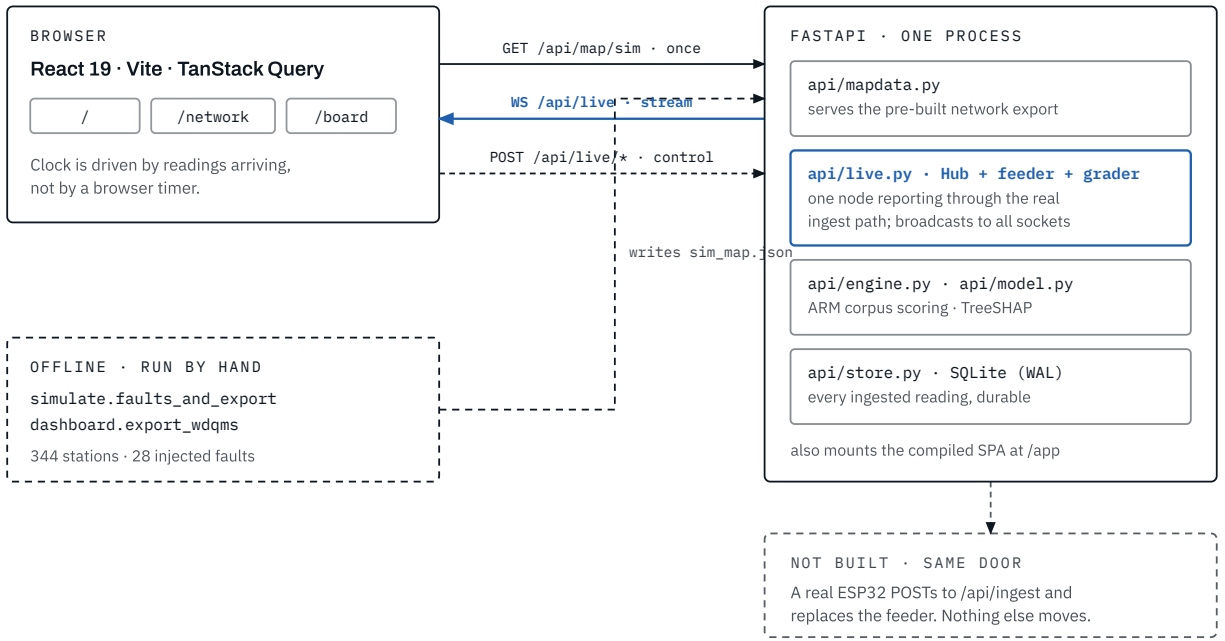
AP 0.406 against a base rate of 0.331 is a lift of 1.22×. Without housekeeping channels the model is barely better than guessing at the base rate, and at a precision an operator would accept it finds essentially nothing. It is served because it is explainable and reproducible, not because it carries the detection. **The neighbour-differencing pipeline of Part II is what works.** Every response from the endpoint carries this caveat.

A further limitation: the model scores **ARM items only**. The simulated export carries no belief state, so `bias_sigma` and `drift_sigma_day` cannot be computed for the Indian stations.

PART IV — AS-BUILT

19 Shape of the system

One FastAPI process serves everything: the JSON exports, the live WebSocket, the model endpoints, and the compiled React app itself. There is no separate frontend host and no database server.



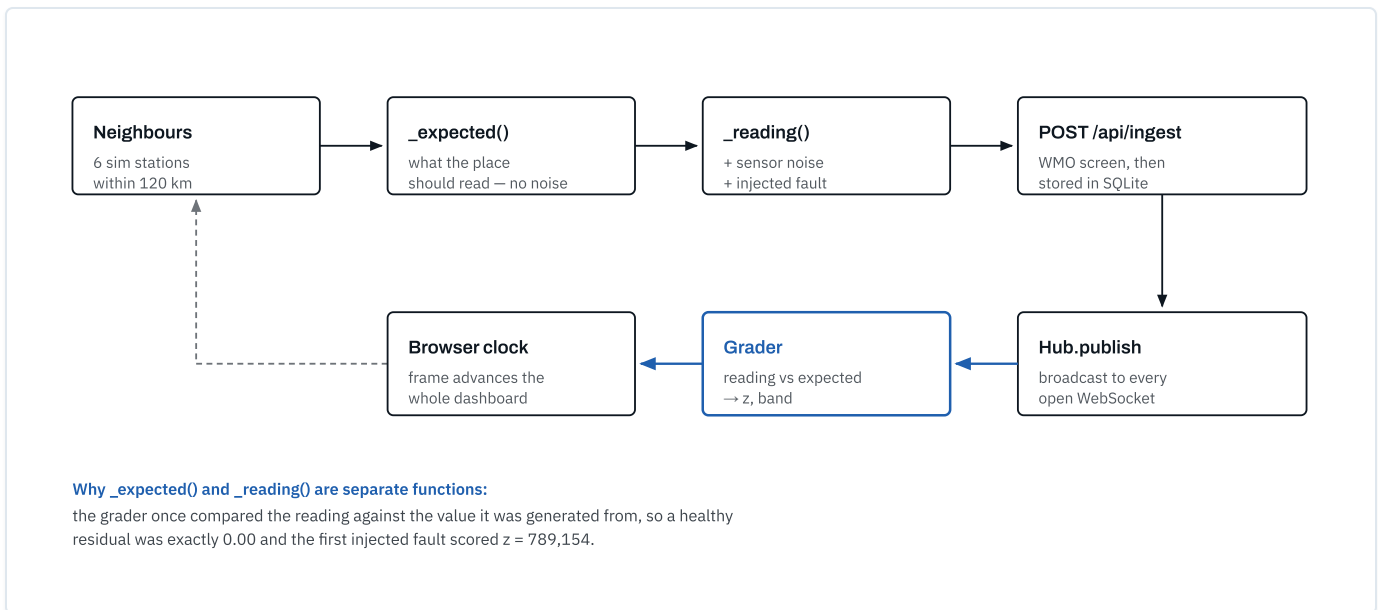
The browser holds three channels to one process. The dashed box on the left is not a service — it is two commands a developer runs to regenerate the JSON the API serves. The dashed box at the bottom is the only piece that hardware changes.

20 The live node

A stand-in for hardware, not a simulation shortcut. It posts through the same `/api/ingest` door an ESP32 would use, and it is graded online, against real neighbours, by the same sigma bands the map uses.

FIELD	VALUE
Station id / name	ESP32-01 · SKYGUARD ESP32
Location	Gandhinagar, Gujarat · 23.2156, 72.6369 · 81 m
Neighbours used	6 simulated stations within 120 km
Simulated step	30 min per reading
Baseline before judging	40 readings — reports "learning" until then

FIELD	VALUE
Sensor noise added	T 0.25 °C · RH 1.2 % · P 0.15 hPa
Injectable faults	drift · offset · stuck · spike · dropout



The node drives the clock. Nothing starts by itself — Play starts it, and only Play.

BEHAVIOUR WORTH KNOWING

Pause and resume continue the record rather than restarting it, because a station that stops transmitting and starts again does not begin its record over. Changing Speed retunes the running node in place via `/api/live/rate` — it does not restart it, which would reset the clock to day 0.

21 Ingest and storage

`POST /api/ingest` is the single door into the system. It applies the WMO-No. 8 Annex 1.A physics screen (`physics/wmo_limits.py`), then writes the reading to SQLite in WAL mode via `api/store.py`. Readings survive a restart; `/api/ingest/history` and `/api/ingest/recent` read them back.

This matters for the demo's credibility: in the real world the data is stored, so it is stored here too. The maintenance board reads ESP32 faults from the same place it reads simulated ones.

22 Module inventory

What is in the serving path, what is offline tooling, and what is research code that no longer runs in production. SERVED means the API imports it at runtime.

MODULE	STATUS	ROLE
api/main.py	SERVED	App, routers, SPA mount, GZip
api/live.py	SERVED	Hub, WebSocket, feeder, online grader, fault injection — 699 lines
api/ingest.py	SERVED	The single ingest door; WMO screen
api/store.py	SERVED	SQLite WAL persistence
api/engine.py	SERVED	ARM scoring; learned detector + conformal
api/model.py	SERVED	Model metadata, scoring, TreeSHAP
api/mapdata.py	SERVED	Serves the pre-built exports
api/tiles.py	SERVED	Map and boundary tiles
api/queue.py	SERVED	Maintenance queue state
api/ai.py	SERVED	Catalog, evaluate, replay
physics/relations.py	SERVED	Magnus, dew point, q, MSL reduction — dependency-free, transliterates to C++
physics/wmo_limits.py	SERVED	WMO-No. 8 Annex 1.A limits
detect/ baseline · features · learned · belief · checkers	SERVED	Imported by the engine

MODULE	STATUS	ROLE
signature/classify.py · health/state.py	SERVED	Fault naming, sensor health
simulate/ network · faults_and_export	OFFLINE	Builds the 344-station network, injects faults, grades, exports
inject/faults.py	OFFLINE	The seven fault models of §14
dashboard/export*.py	OFFLINE	Writes the JSON the API serves
learn/ train · deployable · explain · features	OFFLINE	Trains the models in <code>models/</code>
detect/ trust · allocate · supervised	RESEARCH	Not imported by the API
evaluation/ – 18 modules	RESEARCH	Offline experiments; only <code>metrics.py</code> is imported
firmware/skyguard_node	UNTESTED ON HW	ESP32 + BME280 sketch, Wokwi config
firmware/virtual_node.py	OFFLINE	Same wire protocol in plain Python

23 API surface

GROUP	ENDPOINTS
Live node	WS <code>/api/live</code> · <code>/api/live/replay</code> · <code>/stop</code> · <code>/rate</code> · <code>/seek</code> · <code>/fault</code> · <code>/standing</code> · <code>/station</code> · <code>/status</code>
Ingest	POST <code>/api/ingest</code> · <code>/api/ingest/history</code> · <code>/api/ingest/recent</code>
Network data	<code>/api/map</code> · <code>/api/map/{name}</code> · <code>/api/stations</code> · <code>/api/stations/{station}/series</code>
Model	<code>/api/model</code> · POST <code>/api/model/score</code> · <code>/api/explain</code> · <code>/api/rescore</code>
Alerts & queue	<code>/api/alerts</code> · <code>/api/alerts/{id}</code> · <code>/api/queue</code> · <code>/api/queue/{id}</code>
ARM engine	<code>/api/ai/catalog</code> · <code>/api/ai/evaluate</code> · <code>/api/ai/replay</code>

GROUP	ENDPOINTS
Tiles	/api/tiles/{ver}/{z}/{x}/{y}.png · /api/boundary/... · /api/tiles/status
Meta	/api/summary · /api/meta · /api/clock · /api/snapshot · /healthz · /readyz

`/api/summary` is the authoritative source for every headline number. Do not hardcode these figures anywhere — read them.

24 Frontend

React 19 + Vite + TypeScript, TanStack Query for server state, react-router for routing, Leaflet for the map. Compiled to `web/dist` and mounted by FastAPI at `/app`. Three routes, and no console — the old HTML console was retired.

web/src/	
routes/	HomeRoute · NetworkRoute · BoardRoute
lib/	useLive (WebSocket singleton) · alerts (episodes) · bands · field · sim
api/	client · queries · types · mapTypes · sites
components/	Barograph · BeliefChart · StationChannels · Field · ItemDetail · Badge

`lib/useLive.ts` is the one file worth reading before touching anything live: a module-level socket shared by every subscriber, with reconnect and backoff, holding `byFrame`, `grades`, `frame`, `pass`, `running` and `fault`. The dashboard clock is derived from `live.frame` — **if readings stop arriving, time stops on the page**. That coupling is deliberate, and it is also why several past bugs presented as "the Play button is broken".

25 Deployment

TARGET	LOCATION	NOTES
Live service	Dev-66/skyguard-api	Hugging Face Space, public. Serves API + SPA at <code>/app</code> .

TARGET	LOCATION	NOTES
Source	Devansh-66/skyguard	GitHub, branch <code>main</code>
SDK	Gradio on ZeroGPU	See the caveat below

WHY ZEROGPU FOR A CPU-ONLY PROJECT

A free Hugging Face account cannot create a Docker or cpu-basic Space — the API answers 402. Static Spaces are free but cannot run a detector. ZeroGPU is the only free way to get a Python process. The documented pattern is a no-op `@spaces.GPU` function with all real work outside it; nothing here ever requests the GPU, so no quota is consumed by us or by visitors.

The Space entrypoint launches Gradio with `prevent_thread_lock=True`, then mounts the FastAPI app at `/` — `demo.app.mount("/", api)`. Route order does the work: Gradio keeps its own paths, and the mount catches everything left over. Deleting that one line makes every API route 404 while the Space still reports RUNNING.

26 Not built

Stated plainly, so nobody rediscovers these as surprises.

GAP	STATE	DETAIL
Real MQTT / ESP32 link	NOT BUILT	Firmware sketch exists; never run against real hardware. The feeder occupies its place.
Model scoring for Indian stations	NOT BUILT	The export carries no belief state, so two features cannot be computed.
Authentication	NOT BUILT	Every endpoint is open, including the control POSTs.
Multi-node live tier	NOT BUILT	Exactly one live node exists; <code>_state</code> is a single global.
TimescaleDB / Postgres	NOT BUILT	The blueprint proposes it; storage today is one SQLite file.
Corrected-value output	NOT BUILT	Listed as optional in the problem statement; not attempted.

GAP	STATE	DETAIL
Automated tests	NOT BUILT	Verification is manual, through the browser and the API.

27 Traps already paid for

Each of these cost real debugging time. They are recorded so they cost it once.

TRAP	WHAT ACTUALLY HAPPENS
Differencing raw values	Flags 49% of healthy stations. Remove each station's hourly climatology first (§06).
Sigma fitted in-sample	Inflated 1.61×, 7.6% false alarms. It must be a trailing, lagged rolling MAD (§08).
Skipping the MSL reduction	100 m of elevation is ~11.6 hPa of pure artefact. Every hill station looks broken.
Magnus water coefficients below 0 °C	Manufactures supersaturation at cold and high stations. Select the coefficient set by temperature.
Marking a whole spike window as touched	Depresses every spike metric: most samples in the window were never displaced.
Mounted ASGI apps get no lifespan events	Startup hooks on the mounted FastAPI app never fire on the Space.
<code>CancelledError</code> is a <code>BaseException</code>	<code>suppress(Exception)</code> does not catch it; awaiting a task you just cancelled re-raises through the endpoint.
Awaiting inside a <code>finally</code> during cancellation	The await is cancelled immediately, so a "publish on shutdown" placed there never leaves the process.
Calling a FastAPI endpoint function directly	Defaults arrive as <code>Query</code> objects, not values — comparisons raise inside the task and die silently.
<code>putImageData</code> ignores canvas clip regions	Paint offscreen and <code>drawImage</code> through the clip instead.
The boundary GeoJSON is a bare <code>Feature</code>	Not a <code>FeatureCollection</code> — clipping is skipped silently if you assume otherwise.
A frame high-water mark	<code>Math.max</code> made the clock a ratchet: after a restart it could never go back, and froze permanently.

TRAP	WHAT ACTUALLY HAPPENS
Generic CSS class names on table rows	A row classed <code>live</code> inherited <code>.live { display:flex }</code> and its cells stacked.
<code>pkill</code> on Windows	Does nothing. The old server keeps serving stale code while a second one fails to bind. Kill the port owner found via <code>netstat -ano</code> .

28 Running it

Serve locally

```
python -m uvicorn api.main:app --port 8000
# then open http://localhost:8000/app/network
```

Rebuild the frontend

```
cd web && npm run build      # writes web/dist, which FastAPI mounts at /app
```

Regenerate the network

```
python -m simulate.faults_and_export  # 344 stations, 28 faults -> dashboard/sim_map
python -m dashboard.export_wdqms      # real WQMS departures
```

The node does not start on its own, locally or on the Space. Press Play, or `POST /api/live/replay?per_second=4`.

